Universidad de Córdoba

Ingeniería de Sistemas Software Basados en Conocimiento

Cuaderno de Prácticas

Alumno: Rafael Luque Laplana
Email: i32lular@uco.es
Profesor: Don Antonio Calvo Cuenca
Curso académico: 2025/2026

Córdoba, 3 de marzo de 2026

Índice general

1 Organización Espacio de Trabajo **1**
1.1 Entorno de desarrollo: Spyder 1

2 Práctica 1: Fundamentos de PyQt5 **2**
2.1 Introducción a PyQt5 2
2.2 Bases de PyQt5 2
2.2.1 Creación de ventanas 2
2.2.2 Iconos de aplicación 3
2.2.3 Ayudas emergentes (Tooltips) 4
2.2.4 Creación de botones y Señales 4
2.2.5 Ventanas de mensaje 5
2.3 Menús y barras de herramientas 6
2.3.1 Barra de estado 6
2.3.2 Menú 6
2.3.3 Barra de herramientas 9
2.3.4 Ventana principal completa 9
2.4 Ejercicio final integrador 10
2.4.1 Código completo de la aplicación 11
2.5 Enlaces a los códigos fuente de la Práctica 1 19
2.5.1 Ejemplos básicos (Bases de PyQt5) 19
2.5.2 Ejemplos de menús y barras de herramientas 19
2.5.3 Ejercicio final integrador 19

3 Bibliografia **20**

Índice de figuras

1.1 Interfaz principal de Spyder 1

2.1 Espacio de trabajo en Spyder con la estructura de proyectos de la Práctica 1 2
2.2 Ventana básica creada con simple.py 3
2.3 Ventana centrada en el escritorio 3
2.4 Ventana con icono personalizado en la barra de título 4
2.5 Tooltip emergente mostrando formato HTML 5
2.6 Ventana con botón de salida 5
2.7 Cuadro de diálogo de confirmación al cerrar la ventana 6
2.8 Barra de estado mostrando el mensaje Ready 6
2.9 Menú simple con opción Exit 7
2.10 Submenú Import desplegado dentro del menú File 7
2.11 Menú con opción checkeable marcada 8
2.12 Menú con opción checkeable no marcada 8
2.13 Menú contextual desplegado sobre la ventana 9
2.14 Barra de herramientas con icono de salida 9
2.15 Ventana principal completa con menú, barra de herramientas, widget central y barra de estado 10
2.16 Ventana principal del ejercicio final 17
2.17 Múltiples ventanas de edición abiertas 17
2.18 Ventana de Acerca de en el ejercicio final 18
2.19 Diálogo de confirmación en la aplicación final 18

1 Organización Espacio de Trabajo

1.1. Entorno de desarrollo: Spyder

Para el desarrollo de las prácticas hemos utilizado **Spyder**, un entorno de desarrollo integrado (IDE) gratuito y de código abierto, especialmente orientado a ingenieros y científicos de datos. Spyder ofrece una combinación de edición avanzada, depuración, exploración de variables y visualización, lo que lo hace ideal para el desarrollo de aplicaciones con PyQt5 y manipulación de datos.

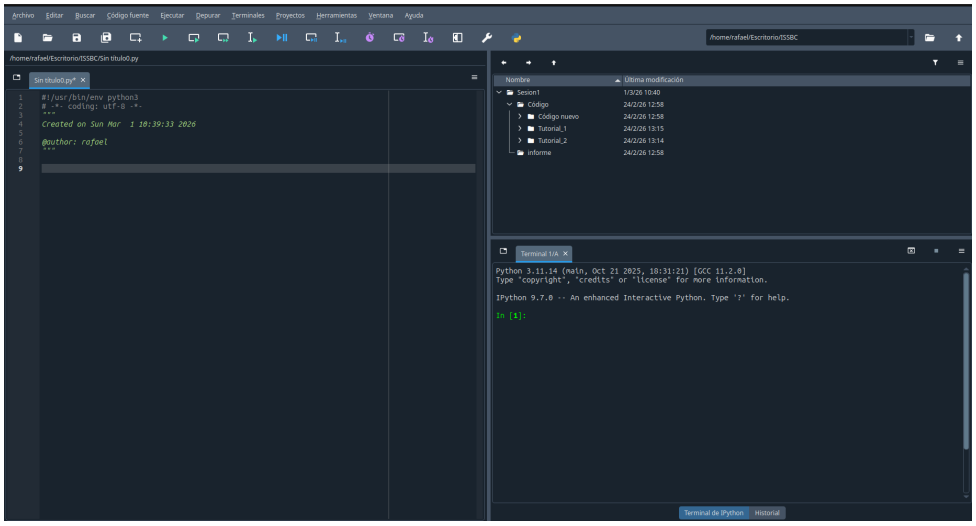

Figura 1.1: Interfaz principal de Spyder

2 Práctica 1: Fundamentos de PyQt5

2.1. Introducción a PyQt5

PyQt5 es un conjunto de bindings de Python para el framework Qt, una biblioteca multiplataforma muy completa para el desarrollo de interfaces gráficas de usuario. En esta práctica, utilizaremos PyQt5 como la herramienta principal para construir las aplicaciones. Su amplia gama de widgets, su sistema de señales y slots, y su madurez lo convierten en una opción ideal para aprender los fundamentos del desarrollo de aplicaciones de escritorio.

Para el desarrollo, utilizaremos el entorno **Spyder**, que forma parte de Anaconda. Spyder nos proporciona un editor integrado, un explorador de variables y una consola interactiva que facilitan la depuración y ejecución de nuestro código.

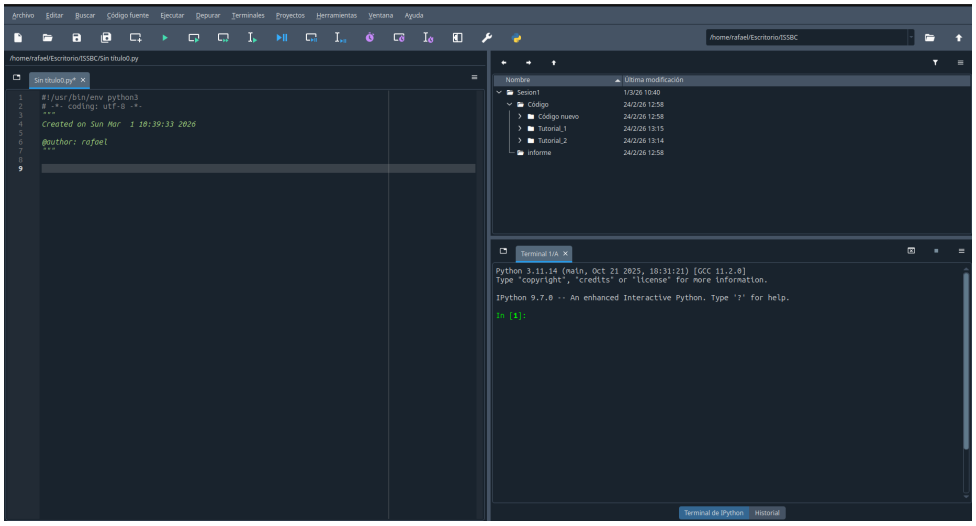

Figura 2.1: Espacio de trabajo en Spyder con la estructura de proyectos de la Práctica 1

2.2. Bases de PyQt5

2.2.1. Creación de ventanas

Ejemplo: `simple.py`

Lo más importante de este primer ejemplo es comprender la **estructura mínima que necesita cualquier aplicación PyQt5**: la creación del objeto `QApplication`, la creación de un widget (en este caso `QWidget` que actúa como ventana), la llamada a `show()` para hacerlo visible y, fundamentalmente, la ejecución del **bucle de eventos** con `app.exec_()`. Sin este bucle, la aplicación se cerraría inmediatamente.

```
app = QApplication(sys.argv)
w = QWidget()
w.show()
sys.exit(app.exec_())
```

Listing 2.1: Estructura mínima de una aplicación PyQt5 (simple.py)

También es relevante observar que un `QWidget` sin padre se convierte automáticamente en una ventana independiente, y que métodos como `resize()`, `move()` y `setWindowTitle()` permiten controlar su apariencia básica.

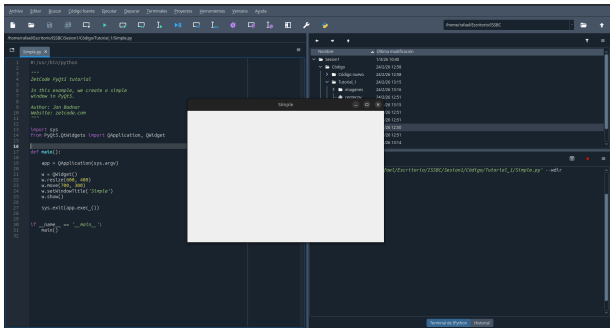

Figura 2.2: Ventana básica creada con simple.py

Ejemplo: `center.py`

La aportación clave de este ejemplo es aprender a **centrar una ventana en la pantalla independientemente de la resolución del monitor**. Para ello se utiliza la clase `QDesktopWidget`, que proporciona información sobre el escritorio.

```
def center(self):
    qr = self.frameGeometry()
    cp = QDesktopWidget().availableGeometry().center()
    qr.moveCenter(cp)
    self.move(qr.topLeft())
```

Listing 2.2: Método para centrar la ventana (center.py)

El método `frameGeometry()` obtiene el rectángulo que ocupa la ventana (incluyendo sus marcos), y `availableGeometry().center()` obtiene el punto central de la pantalla disponible. La combinación de estos métodos permite calcular la posición exacta donde debe situarse la ventana para que aparezca centrada.

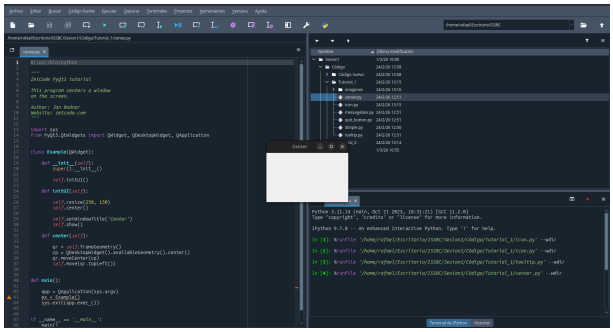

Figura 2.3: Ventana centrada en el escritorio

2.2.2. Iconos de aplicación

Ejemplo: `icon.py`

Lo más relevante de este ejemplo es cómo utiliza la clase `QIcon` para **asignar un icono personalizado a la ventana** mediante `setWindowIcon()`.

```
self.setGeometry(300, 300, 300, 220)
self.setWindowTitle('Icon')
self.setWindowIcon(QIcon('web.png'))
```

Listing 2.3: Asignación de icono a la ventana (icon.py)

También se introduce el método `setGeometry()` que combina posicionamiento y dimensionado en una sola llamada.

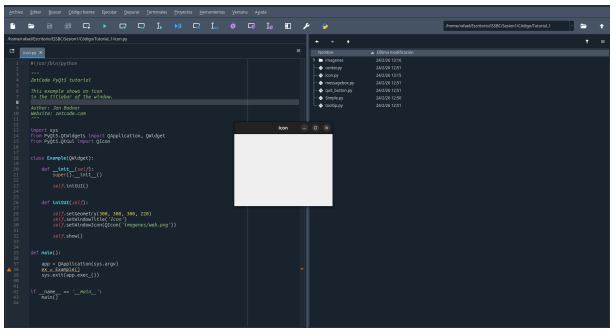

Figura 2.4: Ventana con icono personalizado en la barra de título

Nota sobre la visualización de iconos: Durante el desarrollo de este ejemplo en entornos GNU/Linux con gestores de ventanas modernos (como GNOME o Xfce), es posible que el icono asignado mediante `setWindowIcon()` **no sea visible en la barra de título** de la ventana. Esto no indica un error en el código, sino que se debe a una **decisión de diseño del entorno de escritorio**, que por omisión oculta los iconos para simplificar la interfaz, como es en nuestro caso.

2.2.3. Ayudas emergentes (Tooltips)

Ejemplo: `tooltip.py`

Este ejemplo demuestra cómo **proporcionar información contextual** al usuario mediante tooltips. Lo más interesante es que los tooltips pueden aplicarse a cualquier widget (incluyendo la propia ventana) usando `setToolTip()`, y que admiten **formato HTML** para enriquecer el texto.

```
QToolTip.setFont(QFont('SansSerif', 10))
self.setToolTip('This is a <b>QWidget</b> widget')
btn = QPushButton('Button', self)
btn.setToolTip('This is a <b>QPushButton</b> widget')
```

Listing 2.4: Creación de tooltips (tooltip.py)

Además, se muestra el uso de `sizeHint()` para obtener el tamaño recomendado por el sistema para un botón.

2.2.4. Creación de botones y Señales

Ejemplo: `quit_button.py`

La gran aportación de este ejemplo es la introducción del **mecanismo de señales y slots**, que es el corazón de la comunicación entre objetos en Qt. Aquí vemos cómo la señal `clicked` de un `QPushButton` se conecta al slot `quit` de la aplicación.

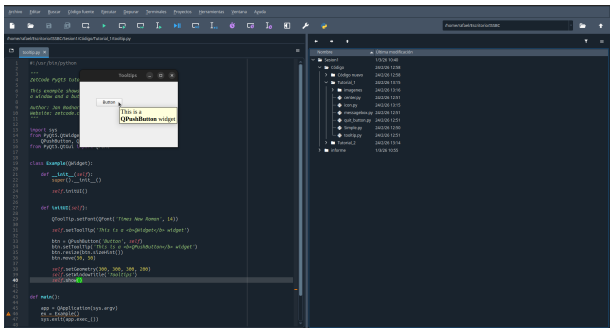

Figura 2.5: Tooltip emergente mostrando formato HTML

```
qbtn = QPushButton('Quit', self)
qbtn.clicked.connect(QApplication.instance().quit)
```

Listing 2.5: Conexión señal-slot para salir de la aplicación (quit$_b$*utton.py*)

Esta conexión es la que hace que al pulsar el botón, la aplicación termine. Es el primer ejemplo de interactividad real.

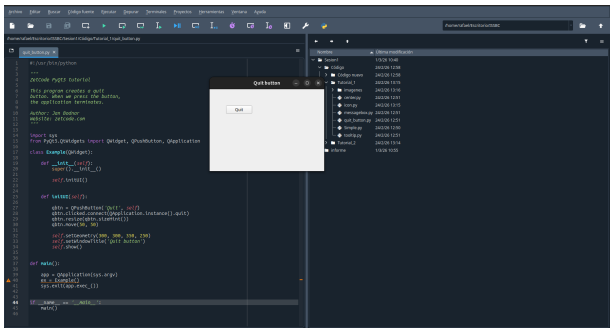

Figura 2.6: Ventana con botón de salida

2.2.5. Ventanas de mensaje

Ejemplo: `messagebox.py`

Lo importante aquí es aprender a **interceptar eventos** (en este caso, el cierre de la ventana) y **mostrar diálogos de confirmación**. Se reimplementa el método `closeEvent()` y dentro de él se crea un `QMessageBox.question()` que presenta al usuario una pregunta con botones "Yesz "No".

```
def closeEvent(self, event):
    reply = QMessageBox.question(self, 'Message',
                                 "Are you sure to quit?",
                                 QMessageBox.Yes | QMessageBox.No,
                                 QMessageBox.No)
    if reply == QMessageBox.Yes:
        event.accept()
    else:
        event.ignore()
```

Listing 2.6: Manejo del evento de cierre con confirmación (messagebox.py)

Dependiendo de la respuesta, se acepta el evento (cerrando la aplicación) o se ignora (cancelando el cierre). Esto permite confirmar acciones críticas como salir sin guardar.

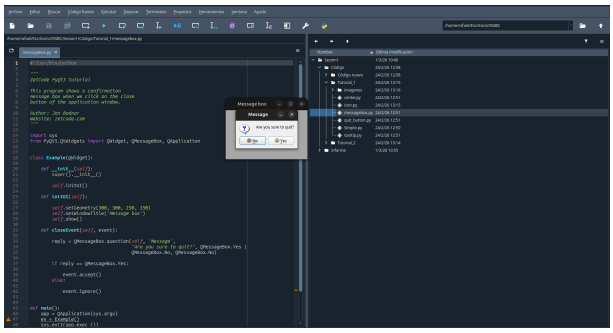

Figura 2.7: Cuadro de diálogo de confirmación al cerrar la ventana

2.3. Menús y barras de herramientas

2.3.1. Barra de estado

Ejemplo: `statusbar.py`

Este ejemplo introduce la clase `QMainWindow`, que proporciona una estructura más completa para aplicaciones, incluyendo soporte nativo para barra de estado, menús y barras de herramientas. Lo más relevante es el método `statusBar()`, que crea (si no existe) y devuelve la barra de estado.

```
self.statusBar().showMessage('Ready')
```

Listing 2.7: Creación de barra de estado (statusbar.py)

El método `showMessage()` muestra un mensaje temporal en la barra de estado, ideal para ofrecer información contextual al usuario.

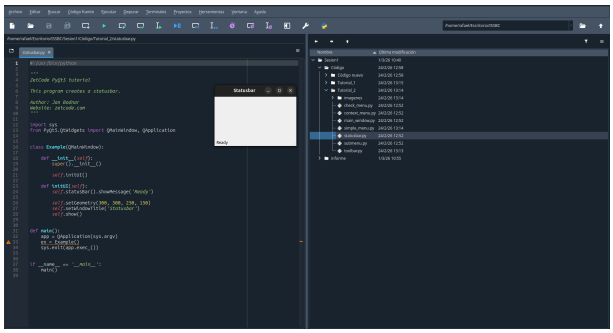

Figura 2.8: Barra de estado mostrando el mensaje Ready

2.3.2. Menú

Menú simple: `simple_menu.py`

Aquí se presenta la clase `QAction`, que es una abstracción fundamental en Qt. Una acción representa un comando que puede estar presente en un menú, en una barra de herramientas o asociado a un atajo de teclado.

```
exitAct = QAction(QIcon('exit.png'), '&Exit', self)
exitAct.setShortcut('Ctrl+Q')
exitAct.setStatusTip('Exit application')
```

```
exitAct.triggered.connect(qApp.quit)

menubar = self.menuBar()
fileMenu = menubar.addMenu('&File')
fileMenu.addAction(exitAct)
```

Listing 2.8: Creación de una acción y su integración en un menú (simple$_menu.py$)

Lo más importante es ver cómo se crea una acción con icono, texto, atajo y mensaje de ayuda, y cómo se añade a un menú.

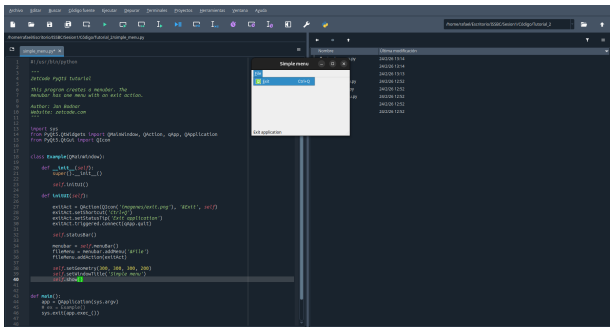

Figura 2.9: Menú simple con opción Exit

Submenú: `submenu.py`

Lo interesante de este ejemplo es la **anidación de menús**. Se crea un objeto `QMenu` independiente para el submenú, se le añaden acciones, y luego se incorpora al menú principal mediante `addMenu()`.

```
impMenu = QMenu('Import', self)
impAct = QAction('Import mail', self)
impMenu.addAction(impAct)
fileMenu.addMenu(impMenu)
```

Listing 2.9: Creación de un submenú (submenu.py)

Esto permite organizar jerárquicamente las opciones cuando hay muchas funcionalidades relacionadas.

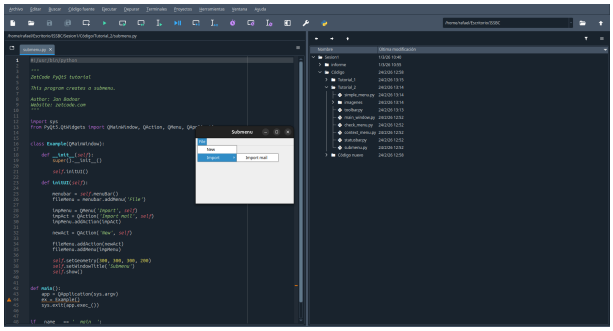

Figura 2.10: Submenú Import desplegado dentro del menú File

Casillas de verificación: `check_menu.py`

La clave de este ejemplo es el uso de **acciones que pueden tener un estado booleano** (activado/desactivado). Al crear una acción con `checkable=True`, esta se comporta como un checkbox en el menú.

```
viewStatAct = QAction('View statusbar', self, checkable=True)
viewStatAct.setChecked(True)
viewStatAct.triggered.connect(self.toggleMenu)
```

Listing 2.10: Acción checkeable para controlar la barra de estado (check$_menu.py$)

Se conecta su señal `triggered` a un método personalizado que muestra u oculta la barra de estado dependiendo del estado de la acción.

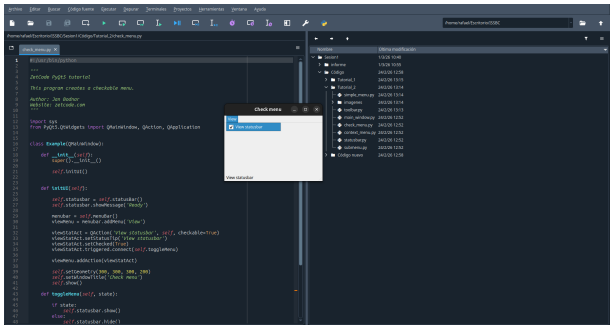

Figura 2.11: Menú con opción checkeable marcada

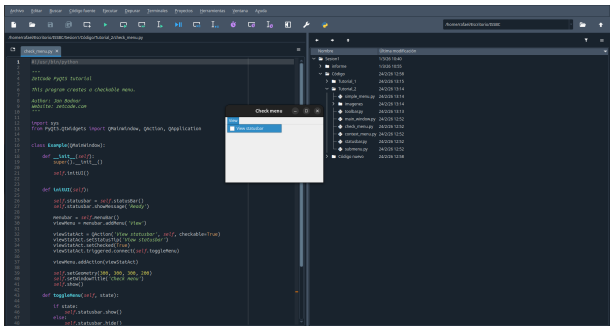

Figura 2.12: Menú con opción checkeable no marcada

Menú contextual: `context_menu.py`

Lo más importante aquí es entender cómo se **crea un menú que aparece bajo demanda** (al hacer clic derecho). Para ello se reimplementa el método `contextMenuEvent()`.

```
def contextMenuEvent(self, event):
    cmenu = QMenu(self)
    newAct = cmenu.addAction("New")
    openAct = cmenu.addAction("Open")
    quitAct = cmenu.addAction("Quit")
    action = cmenu.exec_(self.mapToGlobal(event.pos()))
    if action == quitAct:
        qApp.quit()
```

Listing 2.11: Implementación de menú contextual (context$_menu.py$)

Se crea un `QMenu`, se le añaden acciones, y se muestra con `exec_()` en las coordenadas globales obtenidas mediante `mapToGlobal()`.

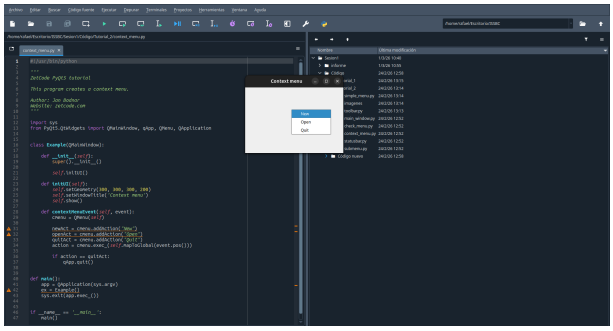

Figura 2.13: Menú contextual desplegado sobre la ventana

2.3.3. Barra de herramientas

Ejemplo: `toolbar.py`

La aportación de este ejemplo es la creación de una **barra de herramientas** mediante `addToolBar()` y la **adición de acciones** a la misma con `addAction()`.

```
self.toolbar = self.addToolBar('Exit')
self.toolbar.addAction(exitAct)
```

Listing 2.12: Creación de barra de herramientas (toolbar.py)

Las barras de herramientas proporcionan acceso rápido a las funciones más comunes mediante iconos, complementando a los menús.

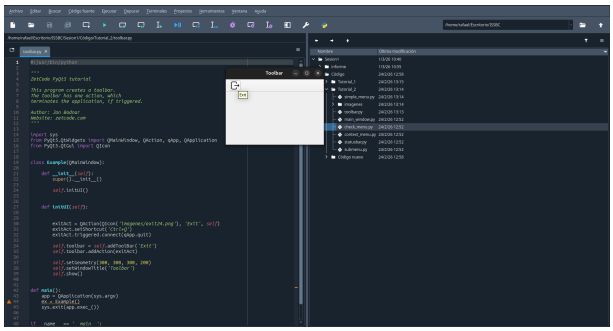

Figura 2.14: Barra de herramientas con icono de salida

2.3.4. Ventana principal completa

Ejemplo: `main_window.py`

Este ejemplo integra todos los elementos anteriores en una **ventana principal completa y funcional**. Lo más relevante es ver cómo se combinan: un menú con acción de salida, una barra de herramientas con la misma acción, una barra de estado y, como novedad, un **widget central** que ocupa el área restante.

```
textEdit = QTextEdit()
self.setCentralWidget(textEdit)
```

Listing 2.13: Configuración del widget central (main$_{w}indow.py$)

Este es el esqueleto básico de cualquier aplicación de edición de texto y sienta las bases para aplicaciones más complejas.

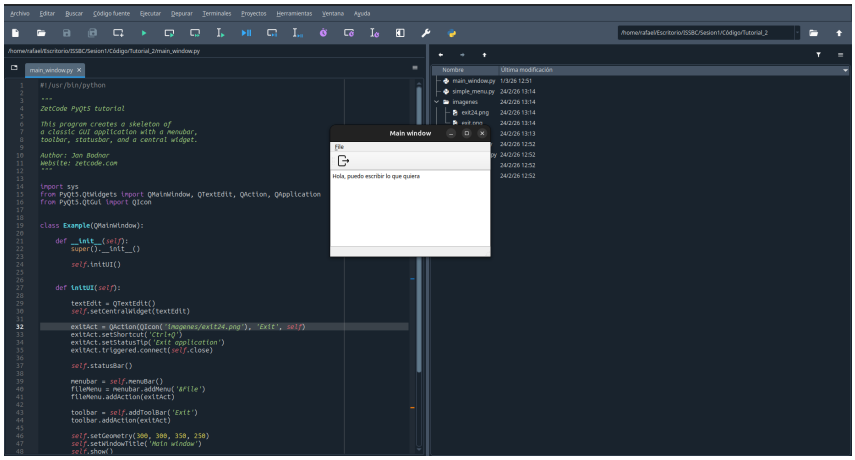

Figura 2.15: Ventana principal completa con menú, barra de herramientas, widget central y barra de estado

2.4. Ejercicio final integrador

Para terminar la práctica, se ha desarrollará una aplicación que integra y demuestra la comprensión de todos los conceptos vistos. Esta aplicación consiste en un **visor de ventanas múltiples** que permite crear nuevas ventanas donde el usuario puede escribir texto e incluye las siguientes características:

- **Ventana principal**: Basada en `QMainWindow`, proporcionando la estructura completa con menús, barra de herramientas y barra de estado.
- **Menús desplegables**:
  - Menú *Archivo*: con opciones para Nueva ventana y Salir.
  - Menú *Ver*: con una opción checkeable para mostrar u ocultar la barra de estado.
  - Menú *Ayuda*: con información sobre la aplicación.
- **Submenú**: Dentro del menú Archivo, un submenú "Nueva"que permite elegir entre "Ventana vacíaz "Ventana con texto".
- **Acciones checkeables**: La opción "Ver barra de estado.^{en} el menú Ver puede marcarse/desmarcarse para controlar la visibilidad de la barra de estado.
- **Menú contextual**: Al hacer clic derecho sobre la ventana principal, aparece un menú con opciones para crear nuevas ventanas.
- **Barra de herramientas**: Contiene iconos para las acciones más frecuentes: Nueva ventana y Salir.

- **Barra de estado**: Muestra mensajes contextuales al pasar el ratón sobre las acciones e información sobre el número de ventanas abiertas.
- **Widget central**: Un `QTextEdit` en la ventana principal donde el usuario puede escribir texto.
- **Ventanas secundarias**: Cada nueva ventana contiene su propio `QTextEdit` independiente.
- **Diálogos de confirmación**: Al seleccionar "Salir", aparece un cuadro de diálogo de confirmación usando `QMessageBox`.
- **Tooltips**: Todos los botones de la barra de herramientas y las acciones principales tienen tooltips descriptivos.

2.4.1. Código completo de la aplicación

A continuación se presenta el código completo de la aplicación final:

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

"""
Ejercicio Final - Pr ctica 1
Aplicaci n integradora de conceptos PyQt5
Autor: Rafael Luque Laplana
Asignatura: ISSBC
"""

import sys
from PyQt5.QtWidgets import (QMainWindow, QTextEdit, QAction, QMenu,
                             QApplication, QMessageBox, qApp, QDialog,
                             QVBoxLayout, QPushButton)
from PyQt5.QtGui import QIcon, QFont
from PyQt5.QtCore import Qt

class VentanaEdicion(QDialog):
    """
    Ventana secundaria de edici n de texto.
    Cada nueva ventana tiene su propio  rea  de texto.
    """

    def __init__(self, titulo="Ventana nueva", texto_inicial=""):
        super().__init__()
        self.setWindowTitle(titulo)
        self.setGeometry(450, 150, 400, 300)

        # Tooltip para la ventana
        self.setToolTip('Ventana de edici n independiente')

        # Layout principal
        layout = QVBoxLayout()

        #  rea  de texto
        self.textEdit = QTextEdit()
        self.textEdit.setText(texto_inicial)
        self.textEdit.setToolTip('Escribe aqu  tu texto')
```

```
        layout.addWidget(self.textEdit)

        # Bot n para cerrar
        btn_cerrar = QPushButton('Cerrar ventana')
        btn_cerrar.setToolTip('Cerrar esta ventana')
        btn_cerrar.clicked.connect(self.close)
        layout.addWidget(btn_cerrar)

        self.setLayout(layout)

class VentanaPrincipal(QMainWindow):
    """
    Clase principal de la aplicaci n.
    Permite crear m ltiples ventanas de edici n.
    """

    def __init__(self):
        super().__init__()
        self.ventanas_secundarias = []  # Lista para mantener referencias
        self.initUI()

    def initUI(self):
        """Inicializa todos los elementos de la interfaz"""

        # Configuraci n b sica de la ventana
        self.setGeometry(300, 300, 600, 400)
        self.setWindowTitle('Visor de Ventanas - Pr ctica 1')

        # Tooltip para la ventana principal
        self.setToolTip('Ventana principal\nCrea nuevas ventanas desde el men ')

        # Intentar establecer un icono (si existe)
        try:
            self.setWindowIcon(QIcon('imagenes/icono.png'))
        except:
            pass

        # Crear todas las acciones
        self.crear_acciones()

        # Crear los men s
        self.crear_menus()

        # Crear la barra de herramientas
        self.crear_barra_herramientas()

        # Crear la barra de estado
        self.crear_barra_estado()

        self.show()

    def crear_acciones(self):
        """Crea todas las acciones QAction utilizadas en la aplicaci n"""

        # --- Acciones del men  Archivo ---
```

```
        # Acci n Nueva ventana vac a
        self.nuevaVentanaAct = QAction('&Ventana vac a', self)
        self.nuevaVentanaAct.setShortcut('Ctrl+N')
        self.nuevaVentanaAct.setStatusTip('Crear una nueva ventana vac a')
        self.nuevaVentanaAct.setToolTip('Nueva ventana vac a (Ctrl+N)')
        self.nuevaVentanaAct.triggered.connect(self.crear_ventana_vacia)
        try:
            self.nuevaVentanaAct.setIcon(QIcon('imagenes/nuevo.png'))
        except:
            pass

        # Acci n Nueva ventana con texto
        self.nuevaVentanaTextoAct = QAction('Ventana con &texto', self)
        self.nuevaVentanaTextoAct.setShortcut('Ctrl+T')
        self.nuevaVentanaTextoAct.setStatusTip('Crear una nueva ventana con texto de ejemplo')
        self.nuevaVentanaTextoAct.setToolTip('Nueva ventana con texto (Ctrl+T)')
        self.nuevaVentanaTextoAct.triggered.connect(self.crear_ventana_con_texto)

        # Acci n Salir
        self.salirAct = QAction('&Salir', self)
        self.salirAct.setShortcut('Ctrl+Q')
        self.salirAct.setStatusTip('Salir de la aplicaci n')
        self.salirAct.setToolTip('Salir (Ctrl+Q)')
        self.salirAct.triggered.connect(self.cerrar_aplicacion)
        try:
            self.salirAct.setIcon(QIcon('imagenes/salir.png'))
        except:
            pass

        # --- Acci n del men  Ver (checkeable) ---

        self.verStatusAct = QAction('Ver &barra de estado', self, checkable=True)
        self.verStatusAct.setChecked(True)
        self.verStatusAct.setStatusTip('Mostrar u ocultar la barra de estado')
        self.verStatusAct.triggered.connect(self.toggle_statusbar)

        # --- Acci n del men  Ayuda ---

        self.acercaAct = QAction('&Acerca de', self)
        self.acercaAct.setStatusTip('Informaci n sobre la aplicaci n')
        self.acercaAct.triggered.connect(self.acerca_de)

    def crear_menus(self):
        """Crea la barra de men  y todos los men s"""

        menubar = self.menuBar()

        # --- Men  Archivo con submen  ---
        fileMenu = menubar.addMenu('&Archivo')

        # Submen  "Nueva"
```

```
        nuevoMenu = QMenu('&Nueva', self)
        nuevoMenu.setToolTip('Crear nueva ventana')
        nuevoMenu.addAction(self.nuevaVentanaAct)
        nuevoMenu.addAction(self.nuevaVentanaTextoAct)

        fileMenu.addMenu(nuevoMenu)
        fileMenu.addSeparator()
        fileMenu.addAction(self.salirAct)

        # --- Men  Ver ---
        viewMenu = menubar.addMenu('&Ver')
        viewMenu.addAction(self.verStatusAct)

        # --- Men  Ayuda ---
        helpMenu = menubar.addMenu('Ay&uda')
        helpMenu.addAction(self.acercaAct)

    def crear_barra_herramientas(self):
        """Crea la barra de herramientas con las acciones principales"""

        toolbar = self.addToolBar('Principal')
        toolbar.setToolTip('Barra de herramientas principal')

        toolbar.addAction(self.nuevaVentanaAct)
        toolbar.addAction(self.nuevaVentanaTextoAct)
        toolbar.addSeparator()
        toolbar.addAction(self.salirAct)

    def crear_barra_estado(self):
        """Crea la barra de estado"""
        self.actualizar_barra_estado()

        # Asignar mensajes de estado a las acciones
        self.nuevaVentanaAct.setStatusTip('Crear una nueva ventana vac a')
        self.nuevaVentanaTextoAct.setStatusTip('Crear una nueva ventana con texto de ejemplo')
        self.salirAct.setStatusTip('Salir de la aplicaci n')
        self.verStatusAct.setStatusTip('Mostrar u ocultar la barra de estado')
        self.acercaAct.setStatusTip('Informaci n sobre la aplicaci n')

    def actualizar_barra_estado(self):
        """Actualiza el mensaje de la barra de estado"""
        num_ventanas = len(self.ventanas_secundarias)
        mensaje = f'Listo - Ventanas abiertas: {num_ventanas}'
        self.statusBar().showMessage(mensaje)

    def contextMenuEvent(self, event):
        """M todo para crear el men  contextual (clic derecho)"""
        cmenu = QMenu(self)

        nuevoMenu = cmenu.addMenu('Nueva ventana')
        nuevoMenu.addAction(self.nuevaVentanaAct)
        nuevoMenu.addAction(self.nuevaVentanaTextoAct)

        cmenu.addSeparator()

```

```
        salirAct = cmenu.addAction('Salir')
        salirAct.triggered.connect(self.cerrar_aplicacion)

        cmenu.exec_(self.mapToGlobal(event.pos()))

    def closeEvent(self, event):
        """Maneja el evento de cierre de la ventana"""
        self.cerrar_aplicacion()
        event.ignore()

    def crear_ventana_vacia(self):
        """Crea una nueva ventana vac a"""
        ventana = VentanaEdicion(f"Ventana {len(self.ventanas_secundarias) + 1}")

        # Conectar se al de cierre para actualizar contador
        ventana.destroyed.connect(self.actualizar_barra_estado)

        ventana.show()
        self.ventanas_secundarias.append(ventana)
        self.actualizar_barra_estado()
        self.statusBar().showMessage('Nueva ventana creada', 2000)

    def crear_ventana_con_texto(self):
        """Crea una nueva ventana con texto de ejemplo"""
        texto = "Este es un texto de ejemplo.\nPuedes modificarlo libremente."
        ventana = VentanaEdicion(f"Ventana {len(self.ventanas_secundarias) + 1}", texto)

        # Conectar se al de cierre para actualizar contador
        ventana.destroyed.connect(self.actualizar_barra_estado)

        ventana.show()
        self.ventanas_secundarias.append(ventana)
        self.actualizar_barra_estado()
        self.statusBar().showMessage('Nueva ventana con texto creada', 2000)

    def toggle_statusbar(self, state):
        """Muestra u oculta la barra de estado seg n el estado del check"""
        if state:
            self.statusBar().show()
            self.actualizar_barra_estado()
        else:
            self.statusBar().hide()

    def acerca_de(self):
        """Muestra informaci n sobre la aplicaci n"""
        QMessageBox.about(self, 'Acerca de',
                          '<h2>Visor de Ventanas - Pr ctica 1</h2>'
                          '<p><b>Autor:</b> Rafael Luque Laplana</p>'
                          '<p><b>Asignatura:</b> ISSBC</p>'
                          '<p><b>Descripci n:</b> Aplicaci n que permite crear '
                          'm ltiples ventanas de edici n independientes.</p>'
```

```
                            '<p><b>Conceptos demostrados:</b></p>'
                            '<ul>'
                            '<li>Ventana principal (QMainWindow)</li>'
                            '<li>Ventanas secundarias (QDialog)</li>'
                            '<li>Men s y submen s</li>'
                            '<li>Acciones checkeables</li>'
                            '<li>Men  contextual</li>'
                            '<li>Barra de herramientas</li>'
                            '<li>Barra de estado</li>'
                            '<li>Tooltips</li>'
                            '<li>Di logos (QMessageBox)</li>'
                            '<li>Se ales y slots</li>'
                            '</ul>')

    def cerrar_aplicacion(self):
        """Cierra la aplicaci n con confirmaci n"""
        num_ventanas = len(self.ventanas_secundarias)

        mensaje = 'Si tiene alguna ventana abierta se cerrar  junto a
    la aplicaci n\n Est s seguro de que quieres salir?'

        reply = QMessageBox.question(self, 'Confirmar salida',
                                     mensaje,
                                     QMessageBox.Yes | QMessageBox.No,
                                     QMessageBox.No)
        if reply == QMessageBox.Yes:
            qApp.quit()

def main():
    app = QApplication(sys.argv)

    ex = VentanaPrincipal()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

Listing 2.14: Ejercicio final integrador (ejercicio_final.py)

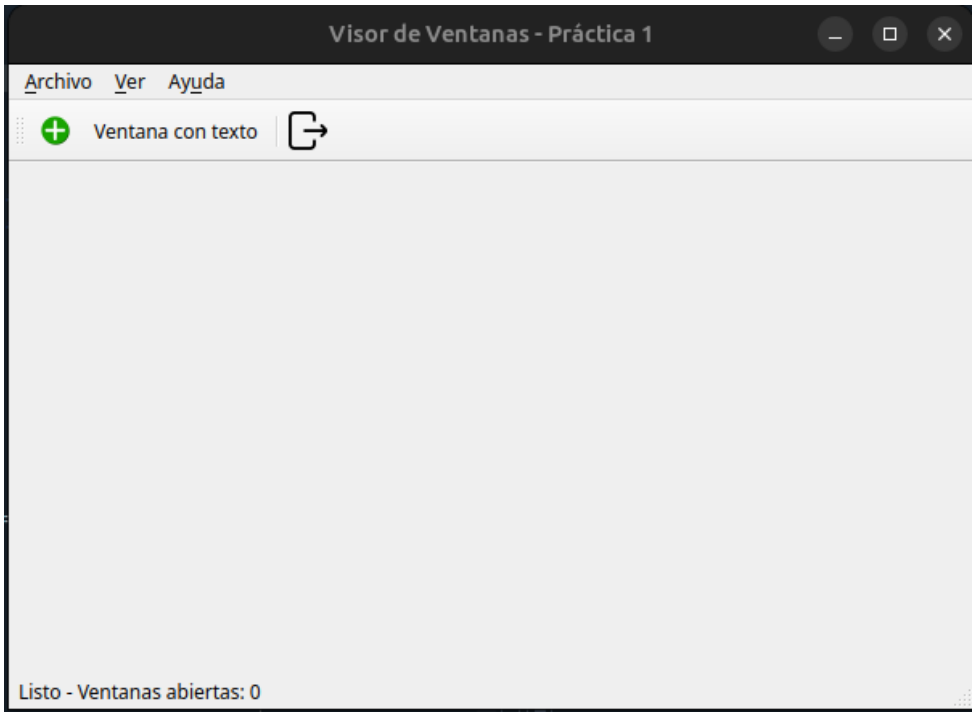

Figura 2.16: Ventana principal del ejercicio final

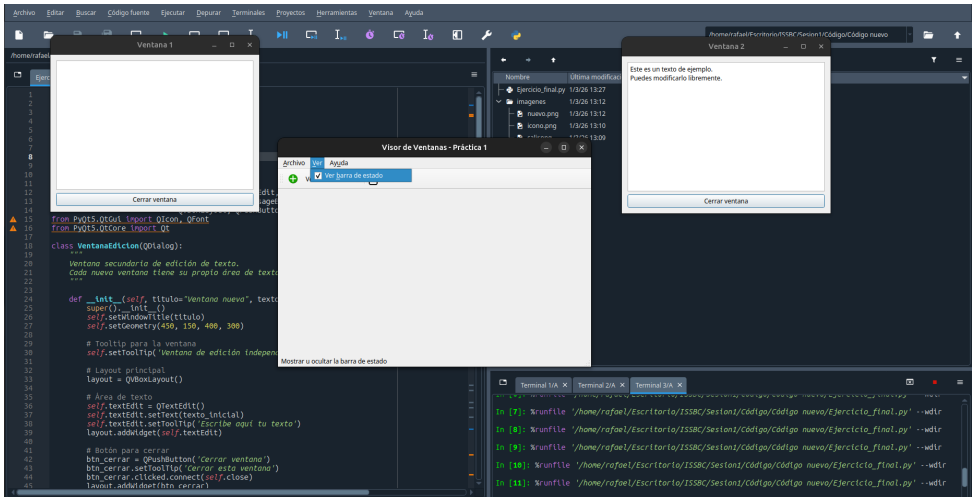

Figura 2.17: Múltiples ventanas de edición abiertas

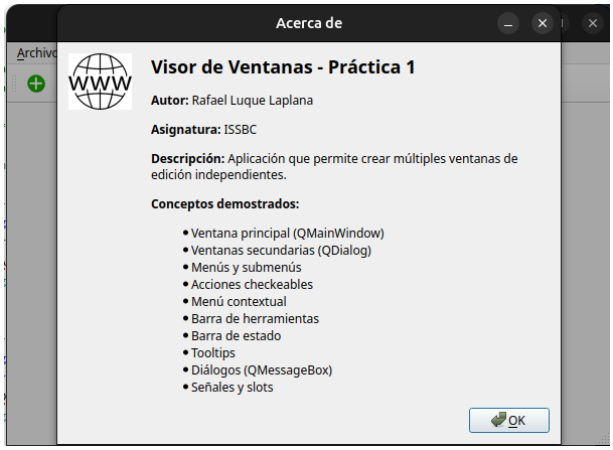

Figura 2.18: Ventana de Acerca de en el ejercicio final

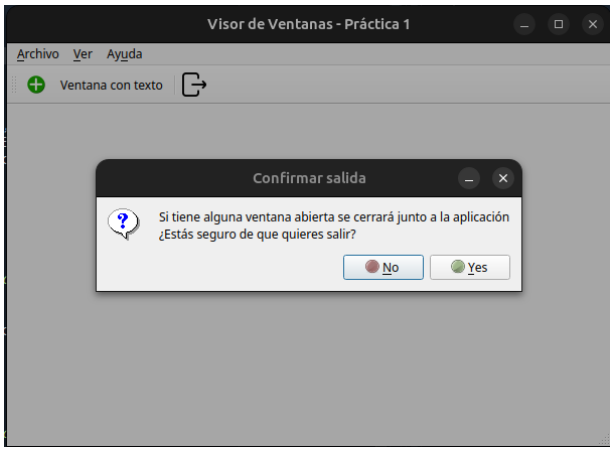

Figura 2.19: Diálogo de confirmación en la aplicación final

2.5. Enlaces a los códigos fuente de la Práctica 1

2.5.1. Ejemplos básicos (Bases de PyQt5)

- **simple.py** - http://www.uco.es/~i32lular/ISSBC/Sesion1/Codigo/Tutorial_1/Simple.py
- **center.py** - http://www.uco.es/~i32lular/ISSBC/Sesion1/Codigo/Tutorial_1/center.py
- **icon.py** - http://www.uco.es/~i32lular/ISSBC/Sesion1/Codigo/Tutorial_1/icon.py
- **tooltip.py** - http://www.uco.es/~i32lular/ISSBC/Sesion1/Codigo/Tutorial_1/tooltip.py
- **quit_button.py** - http://www.uco.es/~i32lular/ISSBC/Sesion1/Codigo/Tutorial_1/quit_button.py
- **messagebox.py** - http://www.uco.es/~i32lular/ISSBC/Sesion1/Codigo/Tutorial_1/messagebox.py

2.5.2. Ejemplos de menús y barras de herramientas

- **statusbar.py** - http://www.uco.es/~i32lular/ISSBC/Sesion1/Codigo/Tutorial_2/statusbar.py
- **simple_menu.py** - http://www.uco.es/~i32lular/ISSBC/Sesion1/Codigo/Tutorial_2/simple_menu.py
- **submenu.py** - http://www.uco.es/~i32lular/ISSBC/Sesion1/Codigo/Tutorial_2/submenu.py
- **check_menu.py** - http://www.uco.es/~i32lular/ISSBC/Sesion1/Codigo/Tutorial_2/check_menu.py
- **context_menu.py** - http://www.uco.es/~i32lular/ISSBC/Sesion1/Codigo/Tutorial_2/context_menu.py
- **toolbar.py** - http://www.uco.es/~i32lular/ISSBC/Sesion1/Codigo/Tutorial_2/toolbar.py
- **main_window.py** - http://www.uco.es/~i32lular/ISSBC/Sesion1/Codigo/Tutorial_2/main_window.py

2.5.3. Ejercicio final integrador

- **ejercicio_final.py** - http://www.uco.es/~i32lular/ISSBC/Sesion1/Codigo/Codigo_nuevo/Ejercicio_final.py

3 Bibliografia

Jan Bodnar, *PyQt5 tutorial - First programs*, ZetCode, 2023.
https://zetcode.com/gui/pyqt5/firstprograms/

Jan Bodnar, *PyQt5 tutorial - Menus and toolbars*, ZetCode, 2023.
https://zetcode.com/gui/pyqt5/menustoolbars/

Anaconda Inc., *Anaconda Documentation*, 2024.
https://docs.anaconda.com/

Spyder IDE, *Spyder Documentation*, 2024.
https://docs.spyder-ide.org/

Riverbank Computing, *PyQt5 Reference Guide*, 2023.
https://www.riverbankcomputing.com/static/Docs/PyQt5/